



PHWriter

License **mit**

build **passing**

coverage **71%**

issues **0 open**

Overview

PHWriter (**PowerShell Help Writer**) is PowerShell module designed to generate beautifully formatted, colored help text for cmdlets and CLI tools, mimicking the style, layout, and readability of modern Linux man pages. It supports modular command architectures (standard cmdlets and router functions), visual layout templates, swappable color theme palettes, **zero-touch AST metadata extraction**, a **TUI interactive pager**, and a **zero-boilerplate CLI router** framework.

Performance Architecture

PHWriter is built from the ground up using strict system automation guidelines:

1. **Root Module Loader:** The main `phwriter.psm1` manages imports using high-performance .NET `[System.IO.Directory]` class bindings, dynamically dot-sourcing private helpers and public cmdlets while strictly controlling public exports via `Export-ModuleMember`.
2. **File-Per-Cmdlet:** Every cmdlet resides in its own isolated `.ps1` script file under the `/Public` directory (e.g. `New-PHWriter.ps1`, `Write-PHAsciiLogo.ps1`).
3. **Encapsulated Helpers:** Shared helper utilities reside under the `/Private` directory.
4. **The .NET First Rule:** Replaces slow PowerShell cmdlets with direct cross-platform .NET library calls (e.g. `[System.Text.StringBuilder]` for string building, `[System.IO.File]` for loading help configs, `[System.Management.Automation.Language.Parser]` for AST extraction).
5. **Color/Layout Exclusivity:** Utilizes a centralized ANSI 256-color parser that applies styling parameters *after* calculating layout padding to prevent character splitting and terminal output

tearing.

Installation

Installation via PowerShell Gallery

```
Install-Module -Name PHWriter -Scope CurrentUser
```

Manual Installation

```
# Clone the repository
git clone https://gitlab.com/phellams/phwriter.git
cd phwriter
Import-Module .\
```

Layout Templates

PHWriter features **6 distinct ASCII/ANSI banner layouts** for rendering module headers:

Layout	Description	Visual Representation
Box	Renders the module name inside a sleek box with rounded corners.	┌...┐
Classic	Renders a retro, double-lined border with text filled in by a custom theme block character.	▬...▬
Minimal	Renders the spaced module name with a simple underlining bar.	___...
Man	Mimics standard Linux manual headers at the top of the command console.	PHW(1) User Commands PHW(1)
Terminal	Renders a terminal prompt icon (>_) on the left, with details on the right.	└ >_ Name

Layout	Description	Visual Representation
Typewriter	Displays a detailed ANSI typewriter graphic on the left, with details on the right.	Typewriter glyph

The 10 Default Themes

PHWriter includes **10 built-in theme configurations** covering diverse visual aesthetics:

1. **default** (Retro Modern): Neon Cyan header and Module name over Dark Green section indicators and Gray meta tags.
2. **matrix** (Cyberpunk Green): High-contrast bright green on black. Perfect for dark terminal hacking setups.
3. **cyberpunk** (Neon Synthwave): Hot Pink accents, Neon Yellow descriptions, and Cyan parameter types.
4. **dracula** (Vampire Theme): Classic Dracula dark theme featuring Purple, Pink, Green, and Yellow highlights.
5. **nord** (Nordic Frost): Clean and calm palette with icy blues, cyan accents, and white text.
6. **Monokai** (`monokai`): Vibrant monokai theme utilizing pink, yellow, green, and orange text highlights.
7. **solarized** (Solarized Dark): Muted cyan and green highlight structure with soft blue parameter names.
8. **sunset** (Golden Hour): Warm reds, orange parameters, and golden yellow highlights.
9. **forest** (Natural Autumn): Forest green, olive green parameters, and gold accents.
10. **classic** (Old-School Man): Monochrome greyscale, mimicking traditional terminal man pages using only bold, underlines, and italics.

Router Command Support

PHWriter can document **router functions**—commands that wrap around multiple subcommands (like `git` or `docker` subcommands). Using the `-Subcommands` array parameter, PHWriter generates a `SUBCOMMANDS` help section detailing the sub-features, syntax, and description of each router endpoint.

Usage Guide

Cmdlet Parameters

New-PHWriter

Generates the help documentation layout:

- **JsonFile** ([string]): File path to a JSON configuration containing help definitions.
- **Name** ([string]): Name of the module/tool to display.
- **CommandInfo** ([hashtable]): Contains `cmdlet`, `synopsis`, `description`, and `source URL`.
- **ParamTable** ([array]): Array of hashtables representing cmdlet parameters (`name`, `param`, `type`, `required`, `description`, `inline`).
- **Subcommands** ([array]): Array of hashtables representing router subcommands (`name`, `syntax`, `description`).
- **Examples** ([string[]]): Array of example execution command blocks.
- **Version** ([string]): Version string. Default: `1.0.0`.
- **Padding** ([int]): Spaces between columns. Default: `3`.
- **Indent** ([int]): Spaces of left indentation. Default: `1`.
- **Theme** ([string\|hashtable]): Theme name (1 of the 10 defaults) or a custom theme hashtable. Default: `'default'`.
- **Layout** ([string]): Banner layout (`Box`, `Classic`, `Minimal`, `Man`, `Terminal`, `Typewriter`). Default: `'Box'`.
- **CustomLogo** ([string]): Custom ASCII logo art.

Write-PHAsciiLogo

Outputs only the themed ASCII logo banner:

- **Name** ([string]): Name of the tool.
- **Version** ([string]): Tool version.
- **Theme** ([string\|hashtable]): Theme configuration.
- **Layout** ([string]): Layout name.

Export-PHWriterMetadata — Alias: phextract

Uses the PowerShell **Language AST parser** to automatically extract all cmdlet metadata from a `.ps1` or `.psm1` source file — eliminating any need for manual JSON/hashtable configuration.

- **Path** ([string], Mandatory): Path to the source script file. Accepts pipeline from `Get-ChildItem`.

- **OutputJson** ([string]): Optional path to serialize the extracted metadata as a `.json` file.
- **ModuleName** ([string]): Override the module name field.
- **Version** ([string]): Override version string. Default: `1.0.0`.
- **Source** ([string]): Documentation URL to embed.

Extracts: function name, synopsis, description, parameter names, types, mandatory flags, aliases, `HelpMessage` attributes, and `.EXAMPLE` comment blocks.



Invoke-PHPager — Alias: `phpager`

A lightweight **interactive TUI pager** rendered inside the terminal. Uses the alternate screen buffer (`\x1b[?1049h`) for clean enter/exit.

- **Content** ([string[]]): Content to display. Accepts pipeline input.
- **Title** ([string]): Header bar title. Default: `PHWriter Pager`.
- **PageSize** ([int]): Override page height. Default: auto from `[Console]::WindowHeight`.
- **NoColor** ([switch]): Strip all ANSI codes before display.

Keyboard controls: `↑/↓` scroll, `PgUp/PgDn` page, `Home/End` jump, `Q/ESC` quit.



New-PHRouter — Alias: `phroute`

A **zero-boilerplate CLI subcommand router** with tab-completion registration and fuzzy-match error recovery.

- **Routes** ([hashtable] , Mandatory): Map of subcommand names $\rightarrow$ [scriptblock] or function name strings. A `'default'` key is invoked when no subcommand is given.
- **ArgumentList** ([string[]]): The raw argument array to dispatch (typically `$args`).
- **ModuleName** ([string]): CLI tool name for output messages. Default: `PHRouter`.
- **RegisterCompleter** ([switch]): Auto-registers a `Register-ArgumentCompleter` for tab completion.
- **Metadata** ([hashtable]): Optional `PHWriter` metadata object for route validation.

Code Examples

Standard Cmdlet Documentation

```
$myCmdletParams = @(
    @{
        Name          = "SourcePath"
        Param         = "s|Source"
        Type          = "string"
        required      = $true
        Description   = "Specifies the source path for the files. Wildcards supported."
        Inline        = $false
    },
    @{
        Name          = "Recurse"
        Param         = "r|Recurse"
        Type          = "switch"
        required      = $false
        Description   = "Indicates that the operation should process recursively."
        Inline        = $true
    }
)

New-PHWriter -Name "MyModule" `
    -CommandInfo @{
        cmdlet      = "Copy-Files"
        synopsis    = "Copy-Files -SourcePath <string> [-Recurse]"
        description = "Copies files from the source path to a predefined directory."
        source      = "https://github.com/myuser/mymodule"
    } `
    -ParamTable $myCmdletParams `
    -Examples @(
        "Copy-Files -SourcePath 'C:\source' -Recurse"
    ) `
    -Version "1.4.2" `
    -Theme "cyberpunk" `
    -Layout "Terminal"
```

Router CLI Command Documentation

```
$subcommands = @(
    @{
        Name      = "init"
        Syntax    = "init [-Force]"
        Description = "Initialize local Git repository configuration."
    },
    @{
        Name      = "commit"
        Syntax    = "commit -Message <string>"
        Description = "Record changes to the repository."
    }
)

New-PHWriter -Name "GitHelper" `
    -CommandInfo @{
        cmdlet      = "git-helper"
        synopsis    = "git-helper <command> [options]"
        description = "A command router function providing simplified Git integrations."
        source      = "https://gitlab.com/githelper"
    } `
    -Subcommands $subcommands `
    -Version "0.9.1" `
    -Theme "matrix" `
    -Layout "Classic"
```

Defining a Custom Theme

You can pass a custom hashtable theme directly to `-Theme` :

```

$myCustomTheme = @{
    AccentColor      = 'red'
    AccentFormat     = 'bold,underline'
    BorderColor      = 'yellow'
    BorderFormat     = 'none'
    HeaderBg         = 'black'
    HeaderFg         = 'white'
    ModuleBg         = 'black'
    ModuleFg         = 'red'
    VersionBg        = 'black'
    VersionFg        = 'yellow'
    SyntaxFg         = 'white'
    SyntaxFormat     = 'none'
    DescriptionFg    = 'white'
    ParamNameFg      = 'red'
    ParamNameFormat  = 'bold'
    ParamTypeFg      = 'yellow'
    ParamTypeFormat  = 'none'
    ParamReqFg       = 'red'
    ParamReqFormat   = 'bold'
    ParamDescFg      = 'gray'
    ExampleFg        = 'yellow'
    DocsFg           = 'red'
    DocsFormat       = 'underline'
    SectionChar      = ' 🔥 '
    HeaderChar       = ' ➡ '
    BorderTop        = ' 🔥 _____ 🔥 '
    BorderBottom     = ' 🔥 _____ 🔥 '
    BorderMiddle     = ' ▤ '
}

```

```
New-PHWriter -Name "FireCLI" -Theme $myCustomTheme -CommandInfo $commandInfo -ParamTable $param:
```

Zero-Touch AST Metadata Extraction

Stop hand-writing JSON configs. Point `Export-PHWriterMetadata` directly at your source files:

```
# Single file – returns a hashtable you can pipe straight into New-PHWriter
$meta = Export-PHWriterMetadata -Path './Public/Get-SystemData.ps1'
New-PHWriter @meta -Theme 'matrix' -Layout 'Terminal'

# Batch extract all Public cmdlets to JSON files for offline caching
Get-ChildItem -Path ./Public/*.ps1 | ForEach-Object {
    Export-PHWriterMetadata `
        -Path $_.FullName `
        -OutputJson "./libs/help_metadata/${($_.BaseName)}.json" `
        -ModuleName 'MyModule' `
        -Version '2.0.0'
}

# Alias shorthand
$meta = phextract ./Public/Invoke-Deploy.ps1
```

TUI Interactive Pager

Pipe any content through `Invoke-PHPager` for arrow-key navigation:

```
# Pipe standard Get-Help output through the pager
Get-Help Get-Process -Full | Out-String | Invoke-PHPager -Title 'Get-Process Help'

# Pipe PHWriter output into the pager for long help docs
New-PHWriter @meta -Theme 'dracula' | Out-String | phpager -Title 'My Module'

# Plain-text mode (no ANSI colors)
Get-Content ./CHANGELOG.md | Invoke-PHPager -Title 'CHANGELOG' -NoColor
```

Zero-Boilerplate CLI Router

Scaffold a full subcommand CLI router in a few lines:

```

function Invoke-MyCLI {
    [CmdletBinding()]
    param(
        [Parameter(ValueFromRemainingArguments)][string[]]$args
    )

    $routes = @{
        'build'    = { param($rest) Invoke-Build @rest }
        'test'     = 'Invoke-PesterTests'
        'release'  = { param($rest) Publish-Module @rest }
        'docs'     = {
            # Extract metadata and display help in pager
            $meta = phextract ./Public/Invoke-MyCLI.ps1
            New-PHWriter @meta -Theme 'nord' | Out-String | phpager -Title 'MyCLI Docs'
        }
        'default' = {
            Write-Host "Usage: mycli <build|test|release|docs>"
        }
    }

    New-PHRouter `
        -Routes          $routes `
        -ArgumentList     $args `
        -ModuleName       'mycli' `
        -RegisterCompleter
}

```

When an unknown subcommand is typed, the router outputs colored **"Did you mean?"** suggestions using Levenshtein distance:

Unknown subcommand: 'buid'

Did you mean?

build

Available subcommands for mycli:

build

docs

release

test

Contributing

1. Fork the Project.
2. Create your Feature Branch: `git switch -c feature/AmazingFeature` .
3. Commit your changes.
4. Push to the branch: `git push origin feature/AmazingFeature` .
5. Open a **Merge Request**.

License

This project is licensed under the MIT License - see the LICENSE file for details.